Paralelismo e OpenMP

Resumo em slides: conceitos básicos, diretivas e avaliação de desempenho

Organizado a partir das anotações de aula

Roteiro

1. **Paralelismo**
   o que é, objetivo, motivação e desafio
2. **Processos e threads**
   conceitos básicos e multithreading
3. **Memória compartilhada × distribuída**
   dois modelos de arquitetura
4. **OpenMP e fork-join**
   API, componentes e modelo de execução
5. **Diretivas de região e tarefas**
   parallel, parallel for, task, single, taskwait
6. **Diretivas de sincronização**
   sections, barrier, master, critical
7. **Cláusula reduction**
   operações all-to-one sem condição de corrida
8. **Avaliação do paralelismo**
   granularidade, speedup e eficiência

O que é paralelismo?

- **Técnica** que divide a computação em partes que podem ser executadas **simultaneamente** por múltiplos núcleos ou processadores
- **Objetivo:** reduzir o tempo de execução ou aumentar o volume de dados processados no mesmo intervalo de tempo
- **Vantajoso** para problemas com grande volume de computação e para problemas que podem ser decompostos em tarefas independentes ou com poucas dependências

Motivação

Potencial de redução do tempo de execução por meio do paralelismo, somado à ampla disponibilidade de processadores **multicore** nos dispositivos computacionais atuais.

Desafio

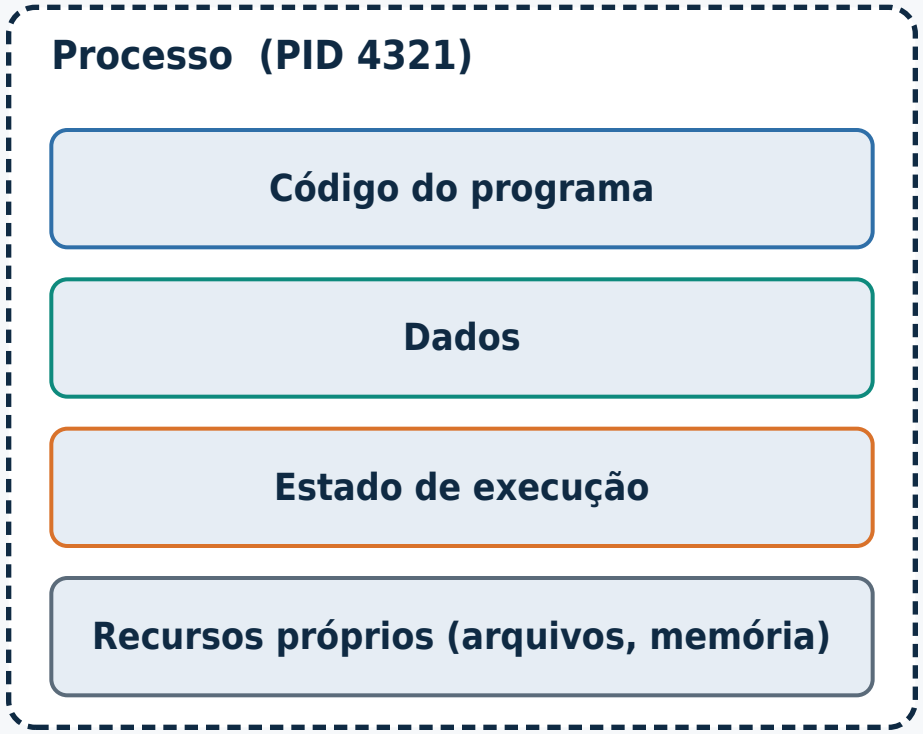
Nem todos os desenvolvedores conhecem os fundamentos da computação paralela. Mesmo entre os que conhecem, aplicar o paralelismo **de forma eficiente** ainda é um desafio.

Overhead = custo extra que você paga para fazer algo funcionar (ex.: criar threads, sincronizar, comunicar). Quando o overhead é maior que o ganho, o paralelismo não compensa.

Conceitos básicos: processos

- Instância de um **programa em execução**
- Criado e gerenciado pelo **sistema operacional**
- Associado a um **usuário proprietário**
- Possui código, dados, estado de execução e **recursos próprios**
- **PID** = número (identificador) do processo
- O comando `top` do Linux mostra todos os processos ativos

> **ex.:** quando clico para abrir o Chrome, o que acontece é que peço ao SO para criar um processo que executa o Chrome.

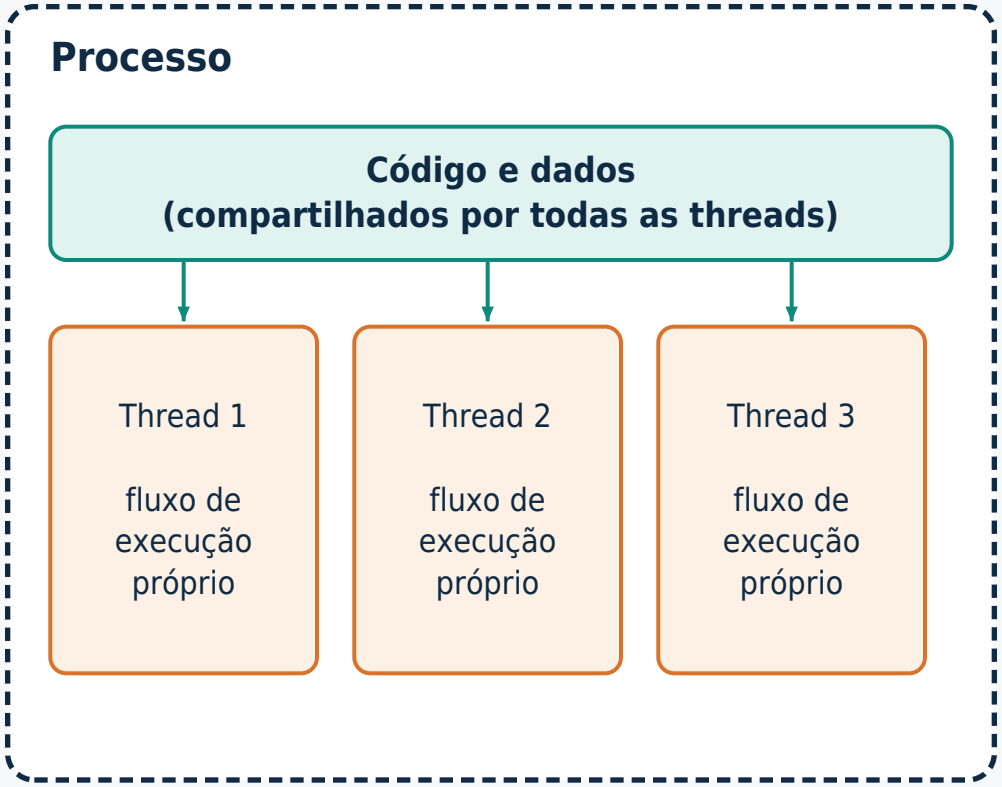


o SO cria e gerencia o processo

Conceitos básicos: threads e multithreading

- **Thread:** fluxo de execução pertencente a um processo
- **Multithreading:** execução de duas ou mais threads em um mesmo processo
- As threads **compartilham** os recursos do processo, como código e dados
- Cada thread possui seu **fluxo de execução independente**

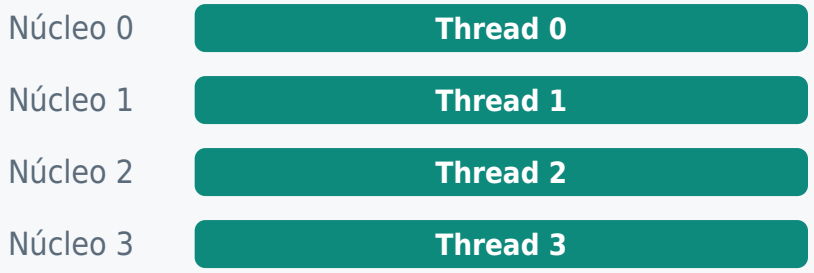
> **ex.:** um processo multithread, como um servidor web multithread (uma thread atendendo cada requisição).



Vantagens das threads

- Compartilham o **espaço de endereçamento** e os recursos do processo, o que facilita a comunicação entre elas. Mas é preciso cuidado para que uma não atrapalhe a outra, e para isso existem **regras** (sincronização)
- **Custos de criação e finalização menores** que os dos processos
- Potencializam o aproveitamento do paralelismo dos processadores **multicore**
- Permitem paralelizar trechos de um fluxo sequencial, **mantendo o restante sequencial**

MELHOR CENÁRIO: colocar uma thread em cada núcleo. Fica bem mais rápido porque há processamento dedicado. Espera-se que o programador tenha dividido o trabalho de forma que todas as threads recebam a **mesma carga** (balanceamento de carga).



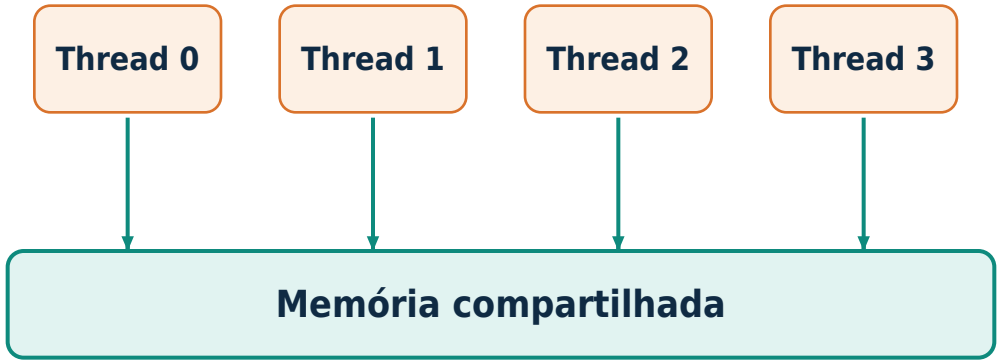
mesma carga em todas as threads

Memória compartilhada × distribuída

Memória compartilhada

As threads acessam um **mesmo espaço de memória**, permitindo a comunicação por meio de **variáveis compartilhadas**.

Modelo do **OpenMP** (um nó multicore).

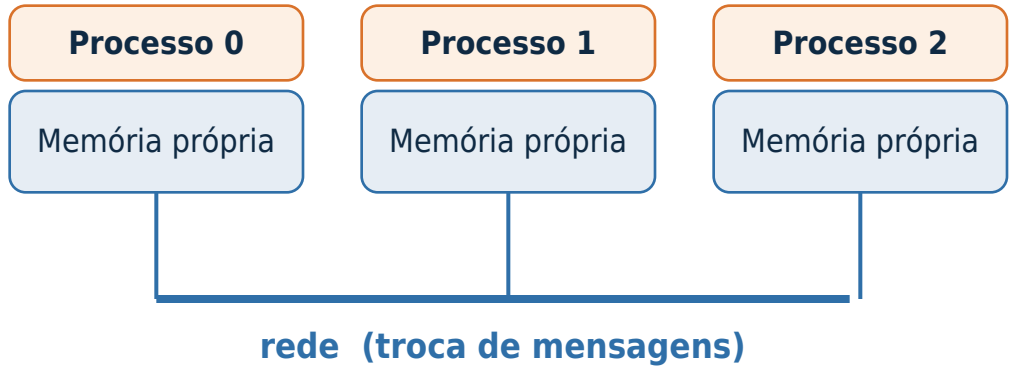


lê e escreve nas mesmas variáveis

Memória distribuída

Cada máquina possui a **sua própria memória**. Os processos da aplicação paralela se comunicam por **troca de mensagens pela rede**.

Modelo típico de clusters (ex.: MPI).



cada processo só enxerga a própria memória

OpenMP

- **API** para programação paralela em arquiteturas de **memória compartilhada**
- Suportada por compiladores de **C, C++ e Fortran**
- Adota o modelo **fork-join** (divisão das tarefas) para a execução paralela
- Composta por três partes:

Diretivas de compilação

instruções para o compilador paralelizar o trecho

```
#pragma omp parallel
```

Rotinas de biblioteca

funções chamadas dentro do programa

```
omp_get_thread_num()
omp_get_num_threads()
```

Variáveis de ambiente

configuram a execução fora do código

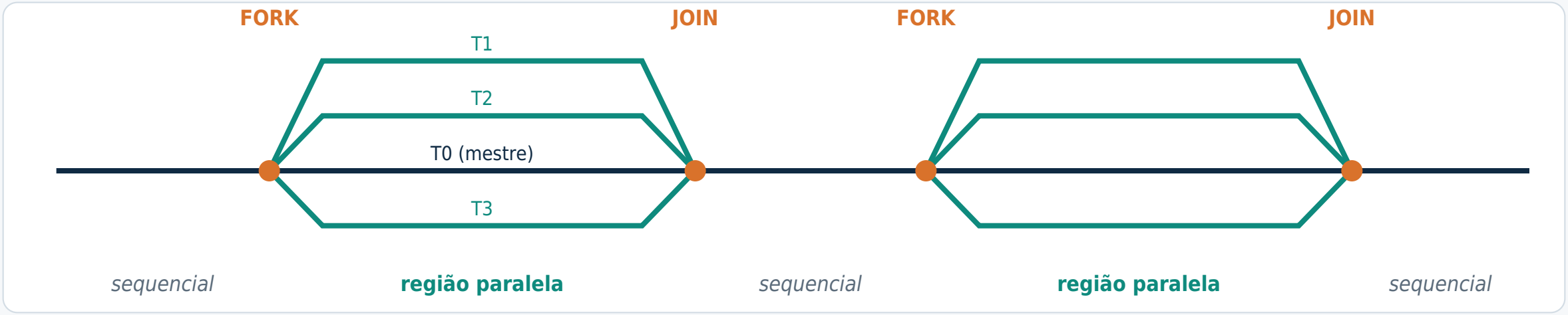
```
OMP_NUM_THREADS=4
```

Como compilar e executar (gcc):

```
gcc -fopenmp programa.c -o programa   // -fopenmp ativa o OpenMP
OMP_NUM_THREADS=4 ./programa         // executa com 4 threads
```

Modelo fork-join

- A execução começa com **uma única thread**: a mestre (thread 0)
- **Fork:** ao entrar em uma região paralela (`#pragma omp parallel`), a mestre cria uma **equipe de threads**; todas executam a região em paralelo
- **Join:** ao final da região há uma **barreira implícita**. As threads se sincronizam e apenas a mestre continua
- Depois do join, o fluxo volta a ser **sequencial** até a próxima região paralela



Diretivas parallel e parallel for

omp parallel

Cria uma **região paralela** na qual **todas** as threads executam o mesmo bloco de código.

```
#pragma omp parallel
{
    printf("Olá da thread %d\n",
           omp_get_thread_num());
}
```

Com 4 threads: **4 linhas** impressas (a ordem não é garantida).

omp parallel for

Cria uma região paralela e **distribui as iterações** de um laço entre as threads.

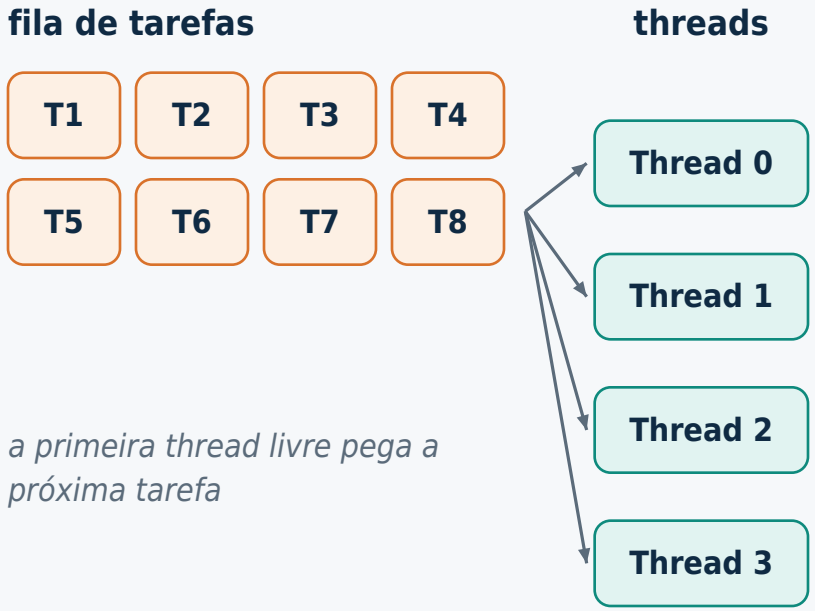
```
#pragma omp parallel for
for (int i = 0; i < 8; i++)
    c[i] = a[i] + b[i];
```

Com 4 threads e divisão estática: cada uma executa ≈ 2 iterações (T0: i=0,1 · T1: i=2,3 · ...).

Atenção: no `parallel for`, as iterações precisam ser **independentes** entre si (uma não pode depender do resultado da outra).

Diretiva omp task

- Cria uma **tarefa** que pode ser executada por **qualquer thread** da equipe
- Permite definir unidades de trabalho cuja execução pode ocorrer de forma **independente**
- Especialmente útil quando as tarefas são **criadas dinamicamente** durante a execução
- Funcionamento: tenho um conjunto de tarefas criadas com `omp task`; a thread disponível pega uma delas
- Se há **mais tarefas que threads**: cada thread pega uma tarefa e a que terminar primeiro pega a próxima, e assim sucessivamente



IMPORTANTE: criar uma tarefa **não garante** que ela será executada imediatamente nem em paralelo. A execução é decidida pelo **ambiente de execução (runtime) do OpenMP**.

Diretiva omp single

- Determina que um bloco de código seja executado por **uma única thread**
- A thread que executa o bloco **não precisa ser a mestre**
- Pode ser usada para que **apenas uma thread crie as tarefas** de uma região paralela
- Garante que duas threads **não executem o mesmo pedaço** ao mesmo tempo
- Ao final do `single` há uma barreira implícita (a menos que se use `nowait`); enquanto esperam, as outras threads podem executar as tarefas criadas

Padrão clássico: parallel + single + task

```
#pragma omp parallel
{
    #pragma omp single
    {
        for (int i = 0; i < N; i++) {
            #pragma omp task
            processa(i);
        }
    }
}
```

> **Por que o single?** sem ele, **cada thread** criaria as N tarefas, e o trabalho seria repetido várias vezes.

Diretiva omp taskwait

- Impõe um **ponto de sincronização**: a tarefa aguarda a conclusão de suas **tarefas filhas** antes de prosseguir
- **Atrasa** o programa, mas em vários casos é necessário (ex.: quando preciso do resultado das filhas)
- Tarefas são especialmente úteis em **algoritmos recursivos**, nos quais novas unidades de trabalho são criadas durante a execução

> **Ligação com granularidade:** em recursões muito profundas (ex.: `fib(2)`), a tarefa é tão pequena que o overhead supera o trabalho. Costuma-se usar um limite (cutoff) e continuar sequencial abaixo dele.

ex.: Fibonacci recursivo com tarefas

```
int fib(int n) {
    int x, y;
    if (n < 2) return n;
    #pragma omp task shared(x)
    x = fib(n - 1);
    #pragma omp task shared(y)
    y = fib(n - 2);
    #pragma omp taskwait  // espera as 2 filhas
    return x + y;
}

int main() {
    int r;
    #pragma omp parallel
    #pragma omp single    // só 1 thread inicia
    r = fib(30);
}
```

Diretivas sections e barrier

omp parallel sections

Segmenta o código em seções (`section`), cada uma executada por **uma única thread**. Útil para trabalhos **diferentes e independentes**.

```
#pragma omp parallel sections
{
    #pragma omp section
    tarefaA();
    #pragma omp section
    tarefaB();
}
```

tarefaA e **tarefaB** rodam ao mesmo tempo, em threads diferentes.

omp barrier

Impõe um **ponto de sincronização**: cada thread **aguarda** até que todas as demais o atinjam.

```
#pragma omp parallel
{
    fase1();
    #pragma omp barrier
    fase2();   // só começa quando
               // todas fecharem a fase1
}
```

Usada quando a fase 2 **depende** do resultado da fase 1 de todas as threads.

Diretivas master e critical

omp master

Determina que um bloco de código seja executado **apenas pela thread mestre** (thread 0). As outras threads **não esperam** (não há barreira implícita).

```
#pragma omp parallel
{
    #pragma omp master
    printf("só a thread 0 imprime\n");
}
```

Diferença para o `single`: no `master` a thread é **sempre a 0**.

omp critical

Cria uma **região crítica**: apenas **uma thread por vez** a executa. Se outra tentar entrar enquanto está em uso, ela **aguarda a liberação**.

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    #pragma omp critical
    total += v[i];  // 1 thread por vez
}
```

Evita **condição de corrida**, mas serializa o trecho. Para somas, prefira **reduction** (próximo slide).

Cláusula reduction

- Aplicada em operações do tipo **all-to-one**: vários valores são combinados em **um único resultado** (soma, produto, máximo...)
- ex.: **soma compartilhada**. Com `reduction`, cada thread trabalha com uma **cópia local** da variável de soma; ao final da região paralela, os valores locais são **combinados** e o resultado vai para a variável compartilhada

> **Qual é o problema?** se todas as threads fizerem `soma += v[i]` na **mesma** variável, duas threads podem ler o mesmo valor antigo e uma atualização se perde (**condição de corrida**).

Como o reduction resolve (3 passos):

1 Fork

cada thread recebe uma cópia privada, iniciada com o **elemento neutro** do operador (0 para +, 1 para ×)

2 Laço

cada thread acumula só na **sua cópia**, sem comunicação nem sincronização durante o laço

3 Join

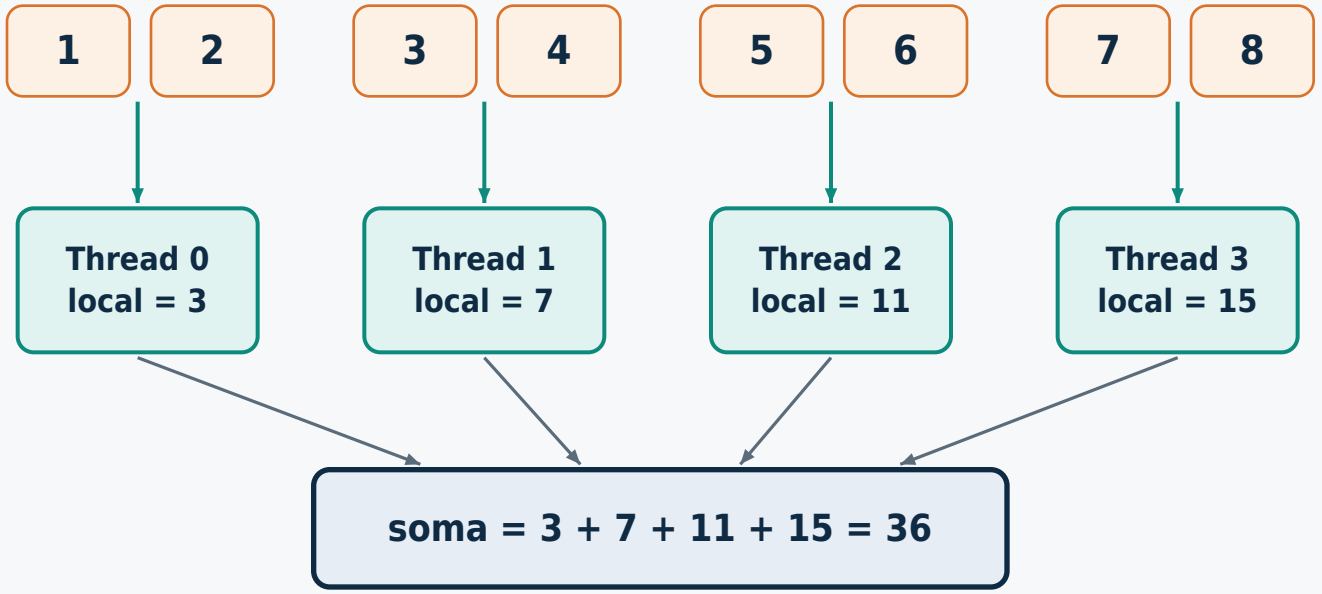
as cópias locais são **combinadas** com o operador e o resultado é guardado na variável original

```
double soma = 0.0;
#pragma omp parallel for reduction(+:soma)
for (int i = 0; i < N; i++)
    soma += v[i];
```

Operadores: `+` `*` `-` `&` `|` `^` `&&` `||` `min` `max`

Reduction na prática

ex.: soma de v = [1, 2, 3, 4, 5, 6, 7, 8] com 4 threads



combinação feita uma única vez, no join

Cada thread só mexe na própria cópia, então não há condição de corrida durante o laço.

A sincronização acontece uma única vez, quando as cópias locais são somadas.

	critical	reduction
Sincronização	a cada iteração	uma vez, no final
Resultado	correto	correto
Desempenho	serializa o trecho (lento)	escala bem com mais threads

Avaliação do paralelismo: granularidade

- **Granularidade:** razão entre o volume de **computação** realizado por uma tarefa e o volume de **comunicação e sincronização** necessários para integrá-la à execução das demais tarefas
- Quanto **maior** a razão, **maior** a granularidade (grossa); quanto **menor**, menor a granularidade (fina)
- Granularidade adequada ajuda a **reduzir o overhead** de comunicação/sincronização e a melhorar o **desempenho** e a **escalabilidade** do sistema paralelo

$$G = \frac{T_{comp}}{T_{com + sinc}}$$

T_{comp} — **tempo de computação útil da tarefa**

$T_{com + sinc}$ — **tempo de comunicação + sincronização**

G alto = granularidade grossa · G baixo = granularidade fina

Granularidade: o que essa razão me entrega?

- A razão responde a uma pergunta simples: **para cada 1 unidade de tempo gasta em overhead, quantas unidades eu gasto trabalhando de verdade?**
- G = 20: **20** de trabalho para cada **1** de overhead. Quase todo o tempo é útil
- G = 0,4: mais overhead do que trabalho (só 0,4 de trabalho para cada 1 de overhead)
- A razão também diz a **fração do tempo realmente útil**: `G / (1 + G)`

> **ex. (pedreiros):** quatro pedreiros levantando um muro. Se a cada tijolo assentado eles precisam parar e avisar os outros (sincronizar), perdem mais tempo conversando do que construindo: **granularidade fina**. Se cada um recebe um pedaço inteiro do muro e só conversa no final: **granularidade grossa**.

Granularidade fina (G baixo)

Tarefas pequenas, muita comunicação/sincronização. O **overhead domina** e o ganho some. Em compensação, a carga fica bem distribuída.

Granularidade grossa (G alto)

Tarefas grandes, pouco overhead. Risco: **desbalanceamento** (uma thread termina antes e fica ociosa) e menos tarefas para distribuir.

Granularidade: como calcular

- **1.** Estimar/medir `T_comp`: tempo de computação útil de uma tarefa
- **2.** Estimar/medir `T_ovh`: tempo de comunicação + sincronização associado a ela (criação, esperas, barreiras, locks)
- **3.** Calcular `G = T_comp / T_ovh` e a fração útil `G / (1 + G)`

Tarefa A
T_comp = 100 ms · T_ovh = 5 ms → G = 20



Tarefa B
T_comp = 2 ms · T_ovh = 5 ms → G = 0,4



> **Leitura:** em A (G = 20) a fração útil é 20/21 ≈ **95%**: vale paralelizar. Em B (G = 0,4) é 0,4/1,4 ≈ **29%**: o overhead come o ganho, então convém **agrupar tarefas** para aumentar a granularidade.

Como aumentar a granularidade no OpenMP:

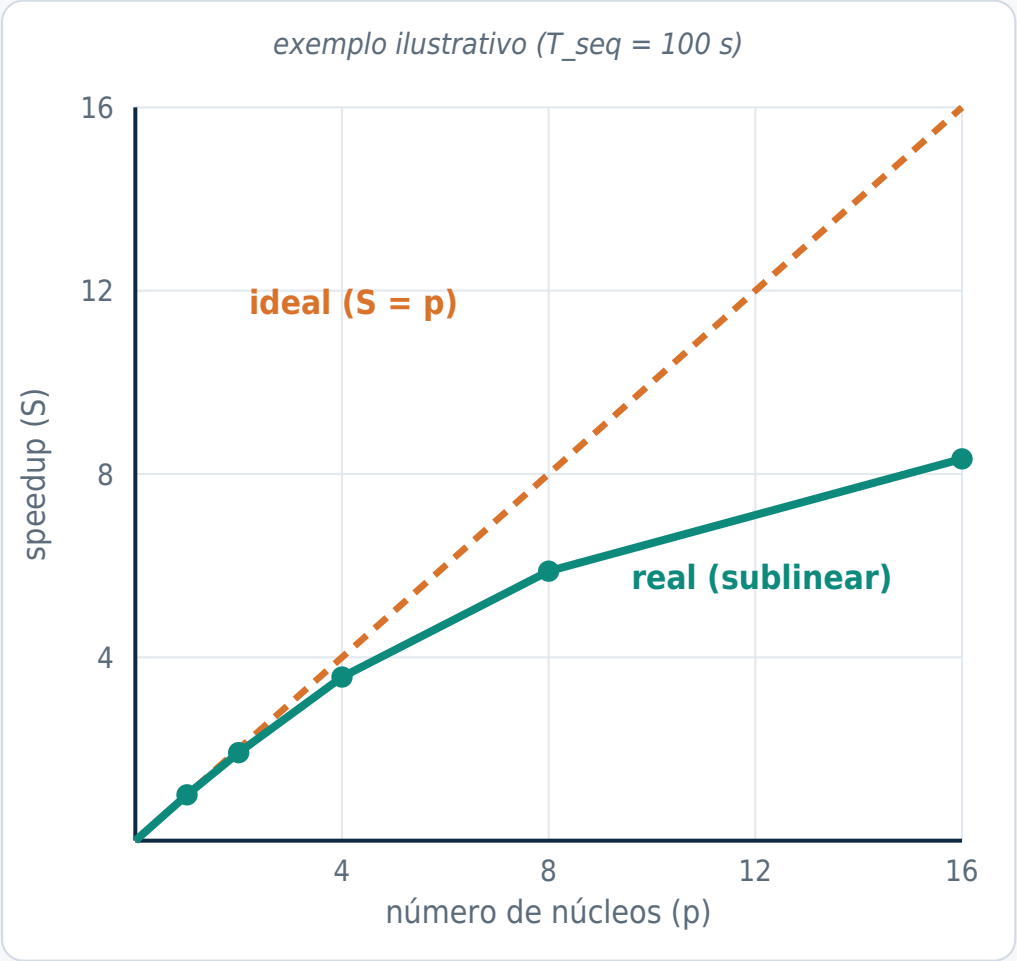
- `parallel for` com blocos maiores de iterações (`schedule(static, chunk)`)
- Tarefas recursivas com limite (cutoff): abaixo dele, continuar sequencial
- `reduction` no lugar de `critical` dentro do laço

Speedup

- Métrica que expressa o **ganho de desempenho** obtido com a paralelização de uma aplicação
- `S = T_seq / T_par` (tempo sequencial ÷ tempo paralelo)
- **S > 1:** ganho de desempenho · **S < 1:** perda de desempenho
- Speedup **ideal** cresce linearmente com o número de processadores (S = p)
- Na prática, o speedup **real** cresce de forma **sublinear**, devido aos custos de comunicação e sincronização

ex.: `T_seq` = 100 s; com 4 núcleos, `T_par` = 28 s → S = 100/28 ≈ **3,57**.

Lei de Amdahl: a fração sequencial f (que não pode ser paralelizada) limita o speedup:
`S ≤ 1 / (f + (1 − f) / p)`



Eficiência

- Métrica que relaciona o **speedup** obtido ao **número de núcleos** utilizados
- `E = S / p = T_seq / (p × T_par)`
- Eficiência **próxima de 1** (100%): **alto aproveitamento** dos núcleos
- Eficiência baixa: núcleos ociosos ou gastando tempo com overhead

> **Tendência:** ao aumentar p, a eficiência costuma **cair**, porque o overhead cresce e a parte sequencial passa a pesar mais.

p	T_par (s)	S	E
1	100	1,00	1,00
2	52	1,92	0,96
4	28	3,57	0,89
8	17	5,88	0,74
16	12	8,33	0,52

ex.: T_seq = 100 s. Barra = eficiência (verde ≥ 0,8)

Resumo: mapa de diretivas e métricas

Diretiva / cláusula	O que faz
`parallel`	cria uma região paralela; todas as threads executam o mesmo bloco
`parallel for`	região paralela + distribui as iterações do laço entre as threads
`task`	cria uma tarefa que qualquer thread da equipe pode executar
`single`	bloco executado por uma única thread (qualquer uma)
`taskwait`	a tarefa espera a conclusão de suas tarefas filhas
`parallel sections`	cada seção do código é executada por uma thread
`barrier`	todas as threads esperam umas pelas outras
`master`	bloco executado apenas pela thread mestre (thread 0)
`critical`	região crítica: uma thread por vez
`reduction(op:var)`	cópia local por thread + combinação final (all-to-one)

Granularidade
$G = T_{comp} / T_{ovh}$

Speedup
$S = T_{seq} / T_{par}$

Eficiência
$E = S / p$